DEPARTMENT: Computer Science and Engineering (CSE), Class [CLASS]
INSTITUTE / COLLEGE: [COLLEGE, CITY]

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: Loan Default Prediction System: Explainable Credit Risk Scoring with Drift Monitoring
Standard Academic Compliance: IEEE Std 830-1998
HCLTech Career Shaper Programme | Project No. 137
Domain: Data Analytics
Program Batch: [PROGRAM BATCH]

SUBMITTED BY:
Learner Name: Lakshay [SURNAME]
Roll Number: [ROLL NUMBER]
Format: Individual Industry Project

Degree: B.Tech, III Year
Academic Year: 2026 - 2027
Admission Number: [ADMISSION NUMBER]

UNDER THE GUIDANCE OF:
Supervisor: [MENTOR NAME]
Industry Partner: HCLTech
Department: Computer Science and Engineering (CSE)

Status: III Year Industry Project Specification
Date of Submission: 01 November 2026

Document Revision & Approval History

Version	Date	Description / Major Changes	Prepared By	Reviewed By
0.1	01-Nov-2026	Initial draft: scope, twelve functional modules, interfaces and design models	Learner	Project guide
1.0	01-Nov-2026	Final III Year SRS baseline, aligned with the project report and the test results of 1 Nov 2026	Learner	Project guide / HOD

> **Document Note:** This Software Requirements Specification follows the IEEE Std 830-1998 recommended practice. It covers the Loan Default Prediction System: the offline Python pipeline that prepares the data, trains and checks the model, and the FastAPI scoring service with its SQLite prediction log and drift monitoring. Requirements and numbers come from README.md, BUILD_LOG.md, the files in reports/ and the source code; test results come from the live API checks and the pytest run of 1 November 2026.

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a complete, clear, and unambiguous definition of the functionality, external interfaces, performance expectations, and constraints of the proposed system: **'Loan Default Prediction System: Explainable Credit Risk Scoring with Drift Monitoring'**. This specification is a reference agreement between the learner, project evaluators, the academic supervisor, and prospective users (loan officers and credit risk analysts) during the 3rd-year engineering curriculum. It is written for the HCLTech Career Shaper Programme, III Year Industry Project No. 137 (Domain: Data Analytics).

1.2 Scope of the System

The proposed system addresses a common lending problem: a lender decides on loan applications with fixed grade rules, which approve some applicants who later default and turn away some who would have repaid. The system estimates each applicant's probability of default (PD) at application time, turns it into APPROVE, REVIEW or REJECT with up to three plain-language reasons, logs every prediction, and raises an alert when new applicants stop looking like the training data. Key boundary guidelines include:

- **In-Scope Capabilities:** A Python pipeline that cleans the Kaggle credit-risk data (32,581 rows, 32,409 after cleaning), splits it 70/15/15, trains logistic regression, random forest and XGBoost and deploys the one with the best validation AUC. Evaluation on 4,862 held-out loans with an NPA back-test against the lender's grade rule, 5-fold cross-validation and an ablation. SHAP explanations. A FastAPI scoring service with four endpoints (/health, /model/info, POST /predict, /monitoring/drift) and Swagger UI. A SQLite prediction log. PSI drift monitoring with an Evidently

HTML report. A kill switch (SCORING_DISABLED=1). A traffic simulator and 13 automated tests.

- **Out-of-Scope Boundaries:** A web front end (Swagger UI is the demo), Docker and container images, cloud hosting, user accounts and login, automatic retraining, a fairness audit across age or other groups, reject inference (the back-test sees only loans that were granted), real lender or credit-bureau data, and any regulatory model validation. No hardware is involved.
- **Expected Benefits:** A ranking that separates defaulters well (test AUC 0.9469 against 0.8614 for logistic regression). At the same 85.5% approval rate as the grade A-C rule, picking applicants by model PD cuts the NPA ratio by amount from 17.75% to 8.12%, which is 54.3% lower. A reason list for every decision (1,201 of 1,201 REVIEW and REJECT test loans). A 95th-percentile response time of 45.6 ms. The data are simulated bureau records with a 21.8% default rate, far above the roughly 1.8% gross NPA ratio of Indian banks, so the relative gain is the useful result, not the absolute NPA figures.

1.3 Definitions, Acronyms, and Abbreviations

Category	Term / Acronym	Definition / Standard Context
Standard	**SRS**	Software Requirements Specification (IEEE Std 830-1998 format).
Credit Risk	**PD (Probability of Default)**	The chance, from 0 to 1, that a borrower does not repay the loan. The model estimates it for each application; the API returns it as probability_of_default.
Credit Risk	**LGD (Loss Given Default)**	The share of a loan lost when the borrower defaults. Not modelled here; the comment on the 0.20 APPROVE cut-off in src/config.py assumes 45% LGD at 11% interest.
Credit Risk	**EAD (Exposure at Default)**	The amount owed when a default happens. Not modelled here; the loan amount (loan_amnt) stands in for it in the NPA ratio.
Credit Risk	**NPA (Non-Performing Asset)**	A loan on which repayment has stopped. The gross NPA ratio by amount used here is defaulted amount divided by amount lent, over approved loans.
Credit Risk	**Decision bands**	APPROVE when PD is below 0.20, REVIEW from 0.20 up to below 0.50, REJECT at 0.50 or above.
Machine Learning	**AUC**	Area under the ROC curve. 0.5 is a coin toss; 1.0 ranks every defaulter above every non-defaulter.
Machine Learning	**Gini**	2 x AUC - 1. Rescales AUC so that 0 is chance and 1 is perfect.
Machine Learning	**KS**	Kolmogorov-Smirnov statistic: the largest gap between the share of defaulters and the share of non-defaulters caught as the cut-off moves.
Machine Learning	**Brier**	Brier score: mean squared difference between the predicted PD and the outcome (0 or 1). Lower is better; it checks that probabilities are honest, not only well ranked.

Category	Term / Acronym	Definition / Standard Context
Machine Learning	**PR-AUC**	Area under the precision-recall curve (average precision). More telling than AUC when defaults are the minority.
Machine Learning	**Stratified split**	A split that keeps the same default rate (0.219) in every part. Here 70/15/15 for train, validation and test, with seed 42.
Machine Learning	**Cross-validation**	Training and scoring on several different cuts of the same data. Here 5 stratified folds; the spread of AUC across the folds shows stability.
Machine Learning	**Pipeline**	A scikit-learn Pipeline: imputing, encoding and the model in one saved object, so training and the API preprocess identically.
Explainability	**SHAP**	SHapley Additive exPlanations. Splits one prediction into one contribution per feature (in log-odds). The contributions plus the base value add up to the model score.
Monitoring	**PSI**	Population Stability Index. Measures how far recent applicants differ from the training data. Below 0.10 stable, 0.10 to 0.25 watch, 0.25 or more alert.
Monitoring	**Kill switch**	The environment variable SCORING_DISABLED=1. Every decision becomes REVIEW (a person decides) while scoring and logging continue.
Architecture	**FastAPI**	Python web framework that serves the scoring API and generates its documentation.
Architecture	**Pydantic**	Python library that checks every request against declared types and ranges. A failed check returns HTTP 422 before the model runs.
Architecture	**Swagger / OpenAPI**	OpenAPI is the machine-readable description of the API. Swagger UI is the page at /docs that shows it and sends test requests.
Architecture	**SQLite**	A database kept in a single file (logs/predictions.db). Holds one row per scored application.
Project Specific	**HCL / HCLTech**	HCLTech Career Shaper Programme, which set this project (Project No. 137).

1.4 References

- **IEEE Std 830-1998:** IEEE Recommended Practice for Software Requirements Specifications.
- **Software Engineering (10th Edition):** Ian Sommerville, Pearson Education.
- **Project Documents:** README.md (setup, endpoints, results), BUILD_LOG.md (every phase with its measured results), reports/evaluation.json, robustness.json, shap_importance.json and drift_summary.csv, models/metadata_v1.json, the console output in _evidence/, the source in src/, scripts/ and tests/, and the HCLTech Career Shaper Programme brief for Project No. 137.

- **Relevant Research Papers & APIs:** Chen, T. and Guestrin, C. (2016), XGBoost: A Scalable Tree Boosting System, KDD 2016. Lundberg, S. M. and Lee, S.-I. (2017), A Unified Approach to Interpreting Model Predictions, NeurIPS 30 (SHAP). Pedregosa, F. et al. (2011), Scikit-learn: Machine Learning in Python, JMLR 12. Siddiqi, N. (2006), Credit Risk Scorecards, Wiley (PSI and scorecard monitoring). Documentation of FastAPI, Pydantic and Evidently. Reserve Bank of India, Fair Practices Code for lenders. Government of India, The Digital Personal Data Protection Act, 2023 (Act No. 22 of 2023).
- **Dataset and Source:** Credit Risk Dataset by laotse on Kaggle, https://www.kaggle.com/datasets/laotse/credit-risk-dataset. Source code: [GITHUB REPOSITORY URL]

1.5 Document Overview

The remainder of this SRS is partitioned into structural sections: Section 2 outlines the product perspective, target user personas, operational context, and design constraints. Section 3 itemizes detailed functional requirements organized into twelve software modules. Section 4 specifies external interfaces (UI, Hardware, Software, Network). Section 5 defines non-functional attributes (Performance, Accuracy, Stability and others) with a measured result and a test method for each, followed by the live checks. Section 6 provides standard academic modeling diagrams: use case, data flow, entity-relationship and sequence. Section 7 is the review and sign-off.

2. Overall Description

2.1 Product Perspective

The project is a self-contained system that runs on one Windows laptop, with no front end, no cloud and no login. It has three parts. (1) An offline Python pipeline (src/data_prep, train, evaluate, robustness, explain and drift) that cleans the data, trains and checks the models and writes the deployed pipeline to models/model_v1.joblib. (2) A FastAPI service (src/api) that loads that file once at start-up, checks each request with Pydantic, scores it, adds SHAP reasons and writes one row per prediction to the SQLite file logs/predictions.db. (3) Drift monitoring that compares the logged applicants with the training data (PSI) and an Evidently report. A lending app is the external caller; in this project Swagger UI and a traffic simulator play that role. Figure 2.1 shows the context (data flow diagram, level 0) and Figure 2.2 the block diagram.

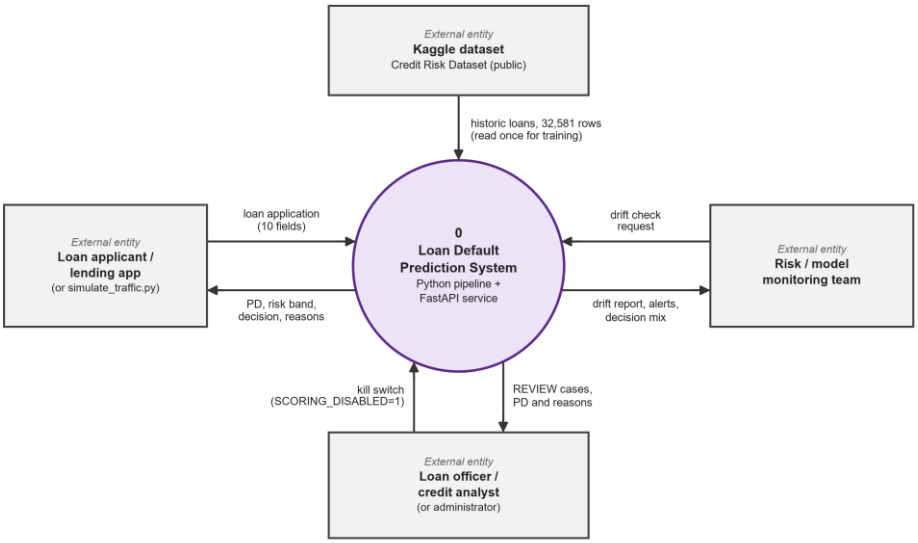


Figure 2.1: System context diagram (DFD level 0). The lending app sends an application (10 fields) and gets PD, risk band, decision and reasons back; the loan officer or credit analyst receives REVIEW cases and can ask for the kill switch; the risk / model monitoring team requests drift checks; the Kaggle dataset is read once for training.

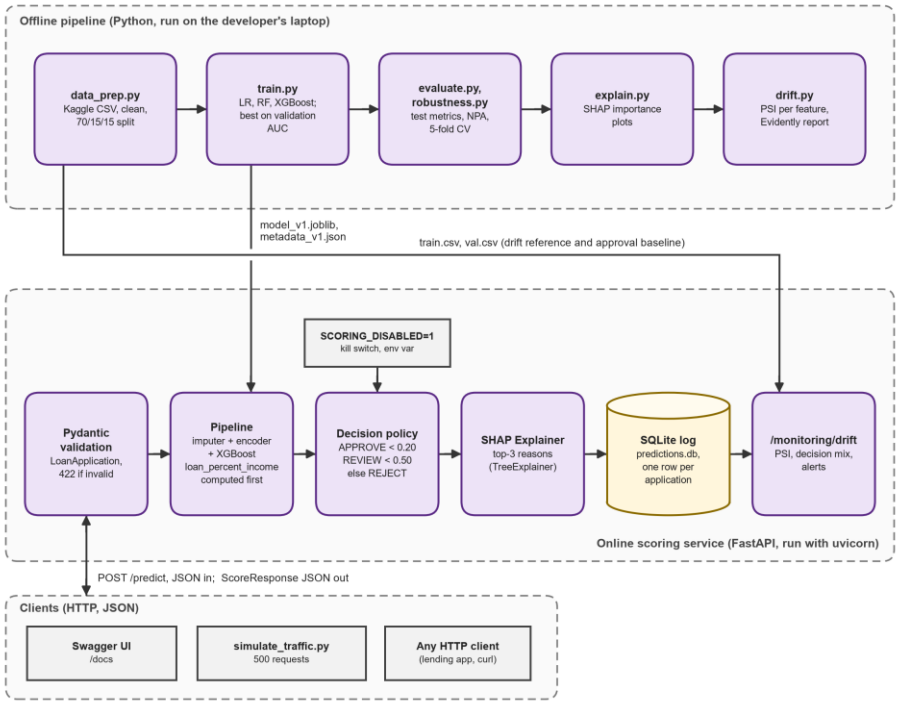


Figure 2.2: Block diagram. The offline pipeline (data_prep to drift) writes the model and the train and validation files; the FastAPI service validates, scores, applies the decision policy and kill switch, explains, logs to SQLite and serves /monitoring/drift. Swagger UI, the simulator and a lending app are the clients.

2.2 User Classes and Characteristics

The target software accommodates distinct user personas. The API has no login, so every caller has the same rights; the personas differ in what they do with it:

User Role / Persona	Technical Proficiency	Core Responsibilities & Rights
Loan Applicant (through a lending app)	None. Does not call the API.	Fills in an application in the lender's app and receives the decision and reasons through it. Appears in the system only as one logged row with no name or contact details.
Loan Officer	Intermediate (credit domain; reads a lending-app screen or JSON)	Handles REVIEW cases: reads PD, risk band and the top reasons, then decides. Can try an application in Swagger UI and see in /health whether the kill switch is on.
Risk Analyst / Model Owner	High (statistics and credit risk)	Reads /model/info, evaluation.json, the NPA back-test, the SHAP charts, the PSI table and the Evidently report. Calls /monitoring/drift. Retrains and evaluates the model by running the pipeline, runs the 13 tests, and decides when the model needs retraining or a cut-off needs a change.
Administrator / ML Engineer	High (Python, Windows command line)	Installs the packages, starts the API with uvicorn, checks /health, sets SCORING_DISABLED when a loan officer or risk analyst asks for manual review, and runs the traffic simulator. Owns the models and logs folders.

2.3 Operating Environment

The application operates within the following software and hardware environments:

- **Client Requirements:** Any HTTP client on the same machine: a web browser for Swagger UI at /docs, PowerShell Invoke-RestMethod, or a script using httpx. No client software is installed. The screenshots in section 4.1 were taken in a desktop browser.
- **Server Runtime Environment:** Windows 10 or 11 with Python 3.11 to 3.14 (built and run on Windows 11 with Python 3.13.14, 16 logical CPUs), the packages pinned in requirements.txt in a virtual environment, and uvicorn serving src.api.main:app on 127.0.0.1:8000. All training and scoring run on the CPU; no GPU is needed.
- **Database Storage Tier:** One SQLite file, logs/predictions.db, created by the API on first start. The data and model are plain files: data/raw and data/processed (CSV), models/model_v1.joblib and metadata_v1.json, and the reports/ folder.

2.4 Design and Implementation Constraints

- **Academic Budget Constraint:** Individual project of four weeks. No paid service: public Kaggle data, open-source Python packages and a laptop. No cloud account or container tooling.
- **Security Compliance:** There is no login, so there are no passwords to store. The service listens on localhost only. The data holds no names or contact details. Every request field is type- and range-checked before the model runs.

- **Modularity Standards:** One scikit-learn Pipeline holds imputing, encoding and the model, so training and the API preprocess identically. Paths, feature lists, seed, model version and cut-offs live in src/config.py. Each phase is one module started with python -m.
- **Data Constraint:** The Kaggle data simulate credit-bureau records. The default rate (21.8%) is far above real bank NPA ratios. The data has no dates, so drift is simulated rather than observed over time. The back-test sees only granted loans. XGBoost results vary slightly with the CPU thread count (test AUC 0.9465 to 0.9478 over 1 to 16 threads).
- **Tool Constraint:** Python with pandas, scikit-learn, XGBoost, SHAP, FastAPI, SQLite and Evidently. No web front end, Docker, cloud hosting or automatic retraining. PSI cut-offs (0.10 and 0.25) are rules of thumb, not tests.

2.5 Assumptions and Dependencies

- **Assumptions:** The calling app sends the ten application fields and not loan_percent_income, which the server computes. The Kaggle file is downloaded by hand to data/raw/credit_risk_dataset.csv. The Kaggle records stand in for a lender's book. A loan officer acts on every REVIEW case. The log holds at least 100 rows before the drift endpoint is used.
- **Dependencies:** Open-source packages pinned in requirements.txt (fastapi 0.141.1, pydantic 2.13.3, scikit-learn 1.8.0, xgboost 3.2.0, shap 0.51.0, evidently 0.7.23). The Kaggle dataset. The Evidently 0.7 API (older module names no longer exist). httpx for the test client and the simulator. A retrained model must keep the same 11 feature names.

3. System Features and Functional Requirements

Requirements are categorized by functional subsystem modules with unique requirement identifiers (FR-x.x). The twelve modules follow the pipeline and the API; the command or route of each is given under its heading. Priority: High = needed for the core purpose in the brief, Medium = expected, Low = convenience. Because the system has no login, the template's authentication module does not apply; access is described in section 2.2.

3.1 Module 1: Data Preparation

Route / command: python -m src.data_prep. Reads the raw Kaggle file, applies three cleaning rules and writes the train, validation and test files to data/processed/.

Req ID	Feature Description	Input / Validation Rule	Priority
FR-1.1	**Load the raw file.** Reads data/raw/credit_risk_dataset.csv (UTF-8) with pandas.	The file must exist. If not, FileNotFoundError names the path and the Kaggle download link.	High
FR-1.2	**Check the columns.** All 12 columns must be present: the 11 features and loan_status (1 = defaulted, 0 = repaid).	A missing column raises ValueError listing the names. Raw file: 32,581 rows, 7,108 defaults (21.8%).	High
FR-1.3	**Remove duplicates.** Exact duplicate rows are dropped because they add no information.	Rules run in a fixed order, this one first. The EDA notebook found 165 duplicate rows.	High

Req ID	Feature Description	Input / Validation Rule	Priority
FR-1.4	**Remove impossible ages.** Rows with person_age above 100 are dropped (data-entry errors such as 123 and 144).	The EDA notebook found 5 rows with age above 100.	High
FR-1.5	**Remove impossible job length, keep gaps.** Rows with person_emp_length above 60 years are dropped. Missing values stay and are filled later, inside the model pipeline.	2 rows above 60 years in the EDA. Gaps: 895 in person_emp_length, 3,116 in loan_int_rate. Result: 32,581 rows become 32,409.	High
FR-1.6	**Stratified split.** 70/15/15 into train, validation and test, stratified on loan_status, seed 42. Written as train.csv, val.csv and test.csv.	22,686 / 4,861 / 4,862 rows, default rate 0.219 in each part. Test: test_splits_are_stratified_and_disjoint.	High

3.2 Module 2: Model Training and Selection

Route / command: python -m src.train. Trains three candidate pipelines on train.csv, scores them on val.csv and saves the winner.

Req ID	Feature Description	Input / Validation Rule	Priority
FR-2.1	**Three candidate pipelines.** Logistic regression (up to 2,000 iterations, balanced class weights), random forest (400 trees, minimum leaf 2, balanced subsample weights) and XGBoost (600 trees, learning rate 0.05, depth 5, subsample 0.8, column sample 0.8).	Seed 42 in every model. n_jobs = -1 (all CPU cores) for random forest and XGBoost; XGBoost test AUC moves between 0.9465 and 0.9478 with the thread count (1 to 16).	High
FR-2.2	**Preprocessing inside the pipeline.** Numeric columns: median imputation (plus standard scaling for logistic regression). Categorical columns: one-hot for logistic regression, one integer code per column for the trees (unknown category = -1).	Fitted on training data only and saved with the model, so the API cannot preprocess differently. No resampling: synthetic rows would distort the probabilities.	High
FR-2.3	**Validation metrics.** For each candidate: AUC, Gini, PR-AUC, KS and Brier (4 decimals) and training seconds.	Validation AUC: logistic regression 0.8789, random forest 0.9368, XGBoost 0.9511.	High
FR-2.4	**Select by validation AUC.** The candidate with the highest validation AUC is deployed. The test set is not touched here.	XGBoost is selected. Model name and version go to the metadata file.	High
FR-2.5	**Save the artefacts.** models/model_v1.joblib (deployed pipeline), models/candidates/*.joblib (all three, for the test report) and models/metadata_v1.json.	Metadata: version, model name, UTC training time, 11 feature names, validation metrics, train rows (22,686) and train default rate (0.2187). Version string from src/config.py.	Medium

3.3 Module 3: Evaluation and Back-Test

Route / command: python -m src.evaluate. Scores the untouched test set once and writes reports/evaluation.json and two charts.

Req ID	Feature Description	Input / Validation Rule	Priority
FR-3.1	**Test metrics for all candidates.** AUC, Gini, PR-AUC, KS and Brier for the three saved pipelines on test.csv.	4,862 loans. Test AUC: XGBoost 0.9469, random forest 0.9291, logistic regression 0.8614. Target for the deployed model: at least 0.85.	High
FR-3.2	**Decision bands.** Share of applicants and actual default rate in APPROVE (PD below 0.20), REVIEW (0.20 up to below 0.50) and REJECT (PD 0.50 or above).	APPROVE 75.3% of applicants (5.0% default), REVIEW 8.3% (26.1%), REJECT 16.4% (97.0%).	High
FR-3.3	**NPA back-test against grade rules.** For each rule (grade A only, A-B, A-C, A-D, all grades) the model approves the same share of applicants, taking the lowest PD first. NPA ratio by amount = defaulted amount divided by amount lent, over approved loans.	A-C rule: 85.5% approved, NPA 17.75% by rule and 8.12% by model, 54.3% lower. Other cuts: A only 95.3%, A-B 78.3%, A-D 9.2%. Target: at least 30% lower.	High
FR-3.4	**Policy comparison.** Approval rate and NPA ratio for three policies: approve everyone, APPROVE band only, APPROVE and REVIEW bands.	Approve everyone 24.93% NPA. APPROVE only: 75.3% approved, 4.87% NPA. APPROVE and REVIEW: 83.6% approved, 6.59% NPA.	Medium
FR-3.5	**Charts and report file.** ROC curves of the three models, NPA ratio against approval rate (11 rates from 30% to 100%), and every number in reports/evaluation.json.	Files: roc_curves.png, npa_vs_approval.png, evaluation.json (UTF-8).	Medium

3.4 Module 4: Robustness Checks

Route / command: python -m src.robustness (about 1 minute). Shows how far the headline result can be trusted. Nothing here changes which model is selected.

Req ID	Feature Description	Input / Validation Rule	Priority
FR-4.1	**Five-fold cross-validation.** Each candidate is scored by stratified 5-fold AUC on all 32,409 cleaned rows. Mean, standard deviation, minimum and the five fold values are saved.	Shuffle seed 7, not the split seed 42. XGBoost 0.9497 ± 0.0024, random forest 0.9336 ± 0.0021, logistic regression 0.8707 ± 0.0059.	High
FR-4.2	**Stability target.** The standard deviation of the fold AUC must stay below 0.01 for every model.	Largest measured: 0.0059 (logistic regression). Deployed model: 0.0024.	Medium
FR-4.3	**Ablation.** XGBoost is retrained without loan_grade and loan_int_rate (the lender's own risk pricing) and scored on the test set.	Test AUC 0.9020. Output: reports/robustness.json.	Medium

3.5 Module 5: Explainability (SHAP)

Route / command: python -m src.explain (batch charts) and the Explainer class that POST /predict calls for every application.

Req ID	Feature Description	Input / Validation Rule	Priority
FR-5.1	**SHAP on the deployed model.** A SHAP TreeExplainer runs on the XGBoost step, fed by the pipeline's own preprocessing, so each original feature has one column. Values are in log-odds; a positive value pushes towards default.	Input: one application or the test set.	High
FR-5.2	**Global importance.** Mean absolute SHAP value per feature on the 4,862 test loans, as a bar chart, a beeswarm chart and shap_importance.json.	Top five: person_income 0.923, loan_grade 0.892, loan_percent_income 0.827, person_home_ownership 0.687, loan_intent 0.558.	Medium
FR-5.3	**Additivity check.** Base value plus the sum of an applicant's SHAP values must equal the model's raw score.	Largest error measured on the test set: 1.34e-05. The automated test limit is 1e-3 on 200 loans.	High
FR-5.4	**Top reasons.** For one application, the features with the largest positive SHAP values, at most 3, largest first. Features with zero or negative values are never listed.	Fewer than 3 reasons (or none) when fewer features push towards default.	High
FR-5.5	**Plain-language text.** Each reason carries a fixed sentence for its feature (11 sentences in src/explain.py), for example "Income is low for the amount requested".	Target: every REVIEW and REJECT case has at least one reason. Measured: 1,201 of 1,201 on the test set.	High

3.6 Module 6: Scoring API

Route / command: POST /predict (JSON body; Swagger UI at /docs). Scores one loan application and returns the decision with its reasons.

Req ID	Feature Description	Input / Validation Rule	Priority
FR-6.1	**Applicant age.** person_age, whole number, required.	18 to 100 inclusive. Age 7 returns HTTP 422 (test T4).	High
FR-6.2	**Income.** person_income, number, required.	Above 0 and at most 10,000,000.	High
FR-6.3	**Employment length.** person_emp_length, years, optional.	0 to 60, or null if unknown. A null is filled with the training median.	Medium
FR-6.4	**Loan amount.** loan_amnt, number, required.	Above 0 and at most 1,000,000.	High

Req ID	Feature Description	Input / Validation Rule	Priority
FR-6.5	**Interest rate.** loan_int_rate, annual percent, optional.	0 to 40, or null if the loan is not priced yet. A null is filled with the training median.	Medium
FR-6.6	**Credit history length.** cb_person_cred_hist_length, whole years, required.	0 to 60.	Medium
FR-6.7	**Category fields.** person_home_ownership, loan_intent, loan_grade and cb_person_default_on_file, all required.	Home ownership: RENT, OWN, MORTGAGE or OTHER. Intent: EDUCATION, MEDICAL, VENTURE, PERSONAL, DEBTCONSOLIDATION or HOMEIMPROVEMENT. Grade: A to G. Default on file: Y or N. Grade Z returns 422.	High
FR-6.8	**Server-side ratio.** loan_percent_income = loan_amnt / person_income, rounded to 2 decimals, computed by the server.	The client does not send it; a value sent anyway is ignored.	High
FR-6.9	**Probability of default.** The saved pipeline gives a PD between 0 and 1, returned rounded to 4 decimals.	Good applicant (age 28, income 42,000, RENT, grade C, loan 12,000 at 13.5%): PD 0.0953.	High
FR-6.10	**Decision and risk band.** PD below 0.20: low and APPROVE. 0.20 up to below 0.50: medium and REVIEW. PD 0.50 or above: high and REJECT.	Cut-offs in src/config.py (APPROVE_BELOW 0.20, REJECT_FROM 0.50). Risky applicant: PD 1.0, high, REJECT.	High
FR-6.11	**Reasons.** top_reasons holds up to 3 items: feature, value, impact (SHAP, log-odds) and text (plain language).	See FR-5.4. Good applicant: RENT +0.267, income 42,000 +0.099, MEDICAL +0.051.	High
FR-6.12	**Response fields.** application_id (UUID), probability_of_default, risk_band, decision, top_reasons, model_version and latency_ms (time spent scoring and explaining, in ms).	Response model ScoreResponse. risk_band: low, medium or high. decision: APPROVE, REVIEW or REJECT.	High
FR-6.13	**Reject invalid input.** A failed rule above, a missing required field or a wrong type returns HTTP 422. Nothing is scored or logged.	Tests: age 7, grade Z and a missing loan_amnt return 422; null employment length and null rate return 200.	High

3.7 Module 7: Health and Model Info

Route / command: GET /health and GET /model/info. Neither takes input.

Req ID	Feature Description	Input / Validation Rule	Priority
FR-7.1	**Health check.** Returns status "ok", model_version and kill_switch (true or false).	Example: status ok, model_version v1, kill_switch false (test T1).	High

Req ID	Feature Description	Input / Validation Rule	Priority
FR-7.2	**Model information.** Returns the contents of models/metadata_v1.json plus the policy cut-offs approve_below 0.2 and reject_from 0.5.	Fields: model_name xgboost, trained_at, 11 features, validation metrics of the three candidates, train_rows 22,686, train_default_rate 0.2187 (test T5).	Medium
FR-7.3	**Load once at start-up.** The pipeline, metadata, SHAP explainer, training data (the drift reference) and the validation approve share are loaded when the server starts, and the SQLite table is created if it is missing.	Nothing is loaded per request. Reference data: data/processed/train.csv. Approve baseline from val.csv: 0.75.	High

3.8 Module 8: Prediction Logging (SQLite)

Route / command: runs inside every POST /predict call. File logs/predictions.db, table predictions.

Req ID	Feature Description	Input / Validation Rule	Priority
FR-8.1	**One row per prediction.** After scoring, one row is written: application_id, created_at (UTC, seconds), model_version, the 10 submitted fields, loan_percent_income, pd, decision and latency_ms.	17 columns. application_id is the primary key. decision is the final decision (REVIEW when the kill switch is on).	High
FR-8.2	**Create on demand.** At start-up the table and an index on created_at are created if they do not exist; the logs folder is created if missing.	CREATE TABLE IF NOT EXISTS and CREATE INDEX IF NOT EXISTS.	Medium
FR-8.3	**Safe connections.** Each write is committed as one unit or rolled back on error, and the connection is always closed.	Closing matters on Windows: an open handle locks the file.	Medium
FR-8.4	**Read recent rows.** The newest n rows (default 500) are read, newest first, for drift monitoring.	Order: created_at descending, then rowid descending.	Medium

3.9 Module 9: Drift Monitoring

Route / command: GET /monitoring/drift?last_n=500 and python -m src.drift. Compares recent applicants with the training data.

Req ID	Feature Description	Input / Validation Rule	Priority
FR-9.1	**PSI for numeric features.** 10 bins whose edges are the training deciles; the outer edges are open; missing values are dropped; an empty bin is floored at 0.0001 so the logarithm never fails.	Seven numeric features: age, income, employment length, loan amount, interest rate, loan-to-income ratio, credit history length.	High

Req ID	Feature Description	Input / Validation Rule	Priority
FR-9.2	**PSI for categorical features.** The share of each category now against its share in training, with the same 0.0001 floor.	Four categorical features: home ownership, loan intent, grade, default on file.	High
FR-9.3	**Status thresholds.** PSI below 0.10 is stable, 0.10 up to below 0.25 is watch, 0.25 or more is alert.	Industry rules of thumb, not statistical tests.	High
FR-9.4	**Score drift.** The same PSI is computed on the model's PD score and added as the row model_score_PD.	12 rows in the table: 11 features and the score.	Medium
FR-9.5	**Drift endpoint.** Reads the last_n most recent logged predictions (integer, default 500) and returns window_size, alerts, decision_mix, approve_baseline and the 12-row table.	Fewer than 100 logged rows return HTTP 409: "Need at least 100 logged predictions, have N" (test T6).	High
FR-9.6	**Alerts and decision mix.** alerts lists every row with status alert. "decision_mix" is added when the APPROVE share differs from the baseline by more than 0.10 (10 points).	Baseline = share of validation loans with PD below 0.20 = 0.75. Downturn run: loan_int_rate (PSI 1.1777) and decision_mix (APPROVE 0.572).	High
FR-9.7	**Offline report.** python -m src.drift compares train.csv with the test set as it is and with a simulated downturn (incomes times 0.8, rates plus 2 points). It writes reports/drift_summary.csv and an Evidently HTML report for the downturn.	Downturn PSI: loan_int_rate 1.1231 (alert), person_income 0.1875 and loan_percent_income 0.1114 (watch). Report: reports/drift_report_downturn.html.	Medium

3.10 Module 10: Kill Switch

Route / command: environment variable SCORING_DISABLED=1, set before uvicorn src.api.main:app starts.

Req ID	Feature Description	Input / Validation Rule	Priority
FR-10.1	**Switch on.** The server reads SCORING_DISABLED once at start-up. The switch is on only when the value is exactly "1".	Any other value, or no variable, means normal operation. A change needs a server restart.	High
FR-10.2	**Every decision goes to a person.** With the switch on, every decision is REVIEW. PD, risk band and reasons are still computed and returned.	Good applicant: PD 0.0953, band low, decision REVIEW (test T11). Automated test: test_kill_switch_sends_everything_to_review.	High
FR-10.3	**Visible and logged.** /health shows kill_switch true. Predictions are still written to the log, with decision REVIEW.	Monitoring continues while nothing is approved or rejected automatically.	Medium

3.11 Module 11: Traffic Simulator

Route / command: python -m scripts.simulate_traffic --n 500 [--downturn] [--url http://127.0.0.1:8000].

Req ID	Feature Description	Input / Validation Rule	Priority
FR-11.1	**Send test-set applicants.** Draws n loans (default 500, seed 1) from test.csv and posts each to /predict without loan_percent_income; missing values become null. Prints round-trip p50 and p95 in ms and the decision mix.	Timeout 10 s per request. Any HTTP error status stops the run. Normal run: p50 35.3 ms, p95 45.6 ms.	Medium
FR-11.2	**Downturn mode.** --downturn applies the stress defined in src/drift.py: incomes times 0.8 (rounded), interest rates plus 2 points, loan_percent_income recomputed.	Downturn run: p95 44.9 ms; APPROVE 0.572, REJECT 0.274, REVIEW 0.154.	Medium

3.12 Module 12: Automated Tests

Route / command: python -m pytest -q tests (13 tests in 3 files).

Req ID	Feature Description	Input / Validation Rule	Priority
FR-12.1	**Data tests (2).** Cleaning gives 32,409 rows with no age above 100, no job length above 60 and no duplicates. Splits have 22,686 / 4,861 / 4,862 rows, default rates within 0.005 of each other and no loan in two parts.	File test_data.py. Needs the raw and processed CSV files.	High
FR-12.2	**Model tests (3).** Test AUC at least 0.85. SHAP values plus the base value equal the raw score within 1e-3. In every PD decile the mean predicted PD is within 0.05 of the actual default rate.	File test_model.py.	High
FR-12.3	**API tests (8).** /health returns ok. A valid /predict gives PD between 0 and 1, a valid decision and at most 3 reasons. Age 7, grade Z and a missing field return 422. Null employment length and rate return 200. The kill switch gives REVIEW. 300 downturn requests make /monitoring/drift list loan_int_rate.	File test_api.py (FastAPI TestClient). Each test writes to its own temporary SQLite file (conftest.py), never to logs/predictions.db. Result: 13 passed (test T12).	High

4. External Interface Requirements

4.1 User Interfaces (UI)

The system has no web front end. Its user interface is Swagger UI, which FastAPI builds from the request and response schemas and serves at http://127.0.0.1:8000/docs. It lists the four endpoints

(/health, /model/info, POST /predict, /monitoring/drift), shows each field's type and allowed values, and has a Try it out button that sends a real request and shows the response. Invalid input shows HTTP 422 with the failing field. The screens below were captured from the running service on 1 November 2026.

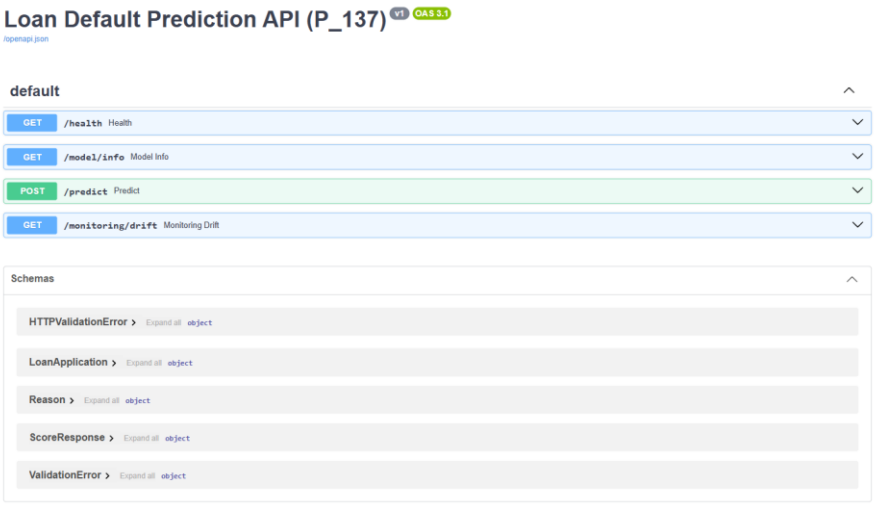


Figure 4.1: Swagger UI of the scoring API. FastAPI generates this interactive documentation at http://127.0.0.1:8000/docs; it lists the four endpoints (GET /health, GET /model/info, POST /predict, GET /monitoring/drift) and the request and response schemas.

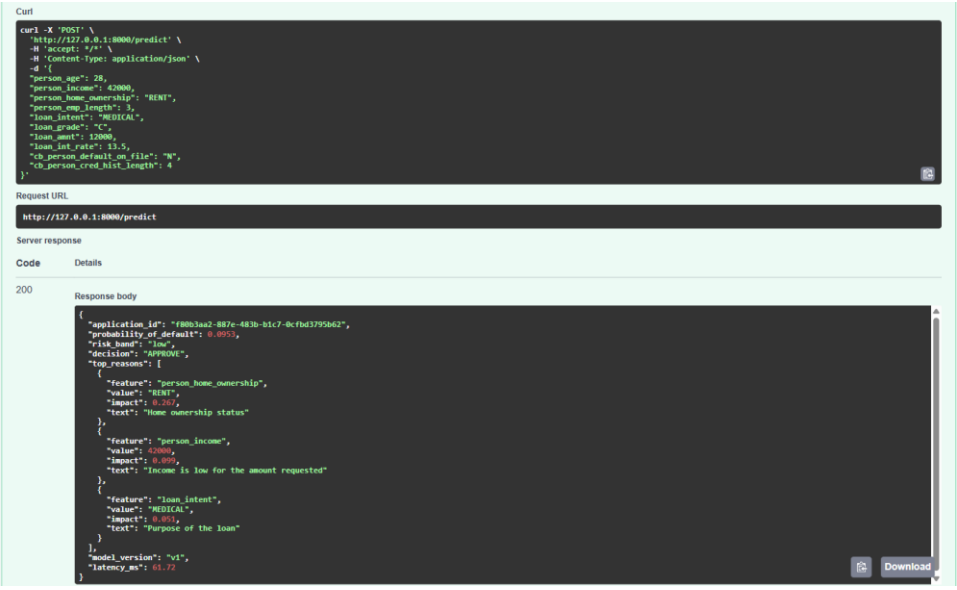


Figure 4.2: Low-risk applicant approved. A 28-year-old renter earning 42,000 asks for 12,000 at 13.5% (grade C): PD 0.0953, risk band low, decision APPROVE, with the three factors that still raised the risk (home ownership RENT +0.267, income +0.099, purpose MEDICAL +0.051).

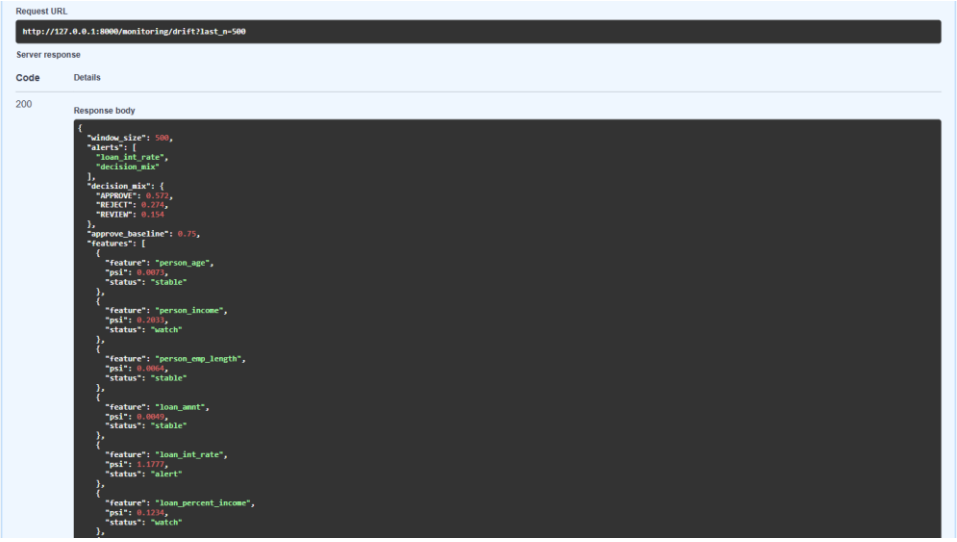


Figure 4.3: Drift alerts after a simulated downturn. After 500 applicants with incomes cut by 20% and rates raised by 2 points, loan_int_rate PSI is 1.1777 (alert), person_income 0.2033 and loan_percent_income 0.1234 (watch), approvals fall to 0.572, and the alerts are loan_int_rate and decision_mix.

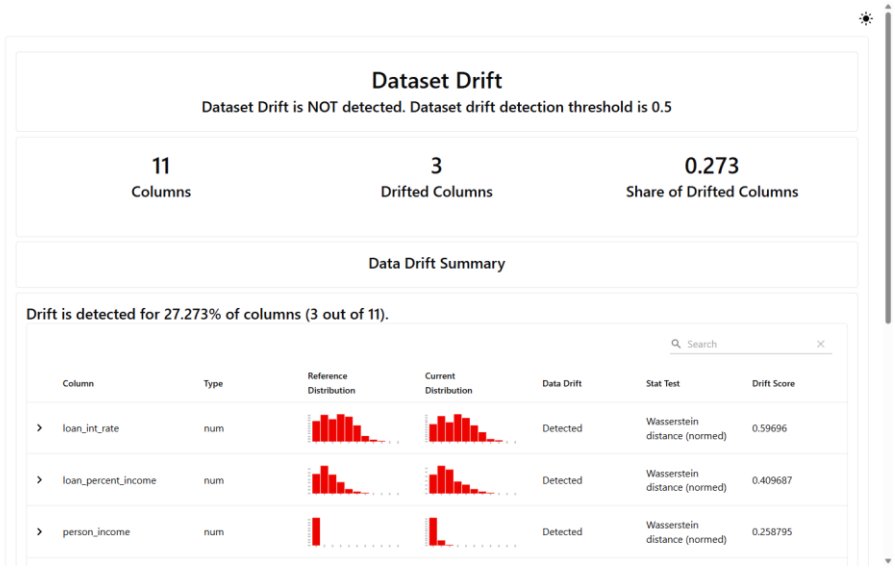


Figure 4.4: Evidently data drift report (downturn). The Evidently 0.7 report (reports/drift_report_downturn.html) compares the stressed test set with the training data: 3 of 11 columns drifted (share 0.273), below its 0.5 dataset-level threshold.

4.2 Hardware Interfaces

Specification of physical interaction points:

- **Standard Deployment:** One Windows laptop or desktop with keyboard, mouse and display. The build machine had 16 logical CPUs. No GPU and no other device is used.

- **Embedded / IoT Variants:** Not applicable. The system has no sensor, microcontroller or other external device.

4.3 Software Interfaces

- **Python Packages:** pandas 3.0.2, numpy 2.4.4, scikit-learn 1.8.0, xgboost 3.2.0, shap 0.51.0, evidently 0.7.23, fastapi 0.141.1, uvicorn 0.46.0, pydantic 2.13.3, starlette 1.10.0, httpx 0.28.1, joblib 1.5.3, matplotlib 3.10.9, seaborn 0.13.2, pytest 9.2.0, all pinned in requirements.txt (Jupyter packages ipykernel 7.4.0 and nbconvert 7.17.2 run the EDA notebook).
- **CSV Interface:** data/raw/credit_risk_dataset.csv (12 columns, 32,581 rows) is read by src/data_prep.py, which writes train.csv, val.csv and test.csv (11 features and loan_status) to data/processed/. Reports are written as JSON, CSV, PNG and HTML in reports/.
- **Model File Interface:** models/model_v1.joblib is one scikit-learn Pipeline (preprocessing and XGBoost) saved with joblib; models/metadata_v1.json describes it; models/candidates/ holds the three candidates for the test report.
- **SQLite Interface:** logs/predictions.db, table predictions with 17 columns, written by src/api/store.py through Python's sqlite3 module and read back as a pandas DataFrame.

4.4 Communication Interfaces

- **Web Traffic Protocol:** HTTP/1.1 with JSON bodies on 127.0.0.1:8000 (uvicorn). Plain HTTP, no TLS, because the service is reachable only from the same machine. The OpenAPI 3 description is served at /openapi.json and the interactive page at /docs. Status codes used: 200, 409 (fewer than 100 logged rows) and 422 (invalid input).
- **Data Serialization:** UTF-8 JSON for requests and responses; null stands for a missing employment length or interest rate. CSV (UTF-8) for the datasets, JSON for model metadata and reports, SQLite for the prediction log.

5. Non-Functional Requirements (NFRs)

Each requirement has a target, a measured result and a test method. "Not measured", "not run" and "not done" mean exactly that: no measurement was made, so no figure is claimed. Results come from the pipeline runs and the live API checks of 1 November 2026 (console output in _evidence/) and from the 13 pytest tests.

Attribute	Target Metric / Standard	Verification & Testing Method
5.1 Performance	Round-trip time of POST /predict, 95th percentile, below 100 ms. **Measured: 45.6 ms** (p50 35.3 ms) over 500 normal requests and 44.9 ms over 500 downturn requests. A single call varies (latency_ms 24.01 and 34.19 ms in T2 and T3, 61.72 ms in Figure 4.2), so the p95 over many requests is the measure. The model loads once at start-up, not per request.	python -m scripts.simulate_traffic --n 500 times every request (client and server on one Windows 11 laptop, 16 logical CPUs). T7 and T9. Not measured: parallel callers, other hardware.

Attribute	Target Metric / Standard	Verification & Testing Method
5.2 Accuracy	Test ROC-AUC at least 0.85. **Measured: 0.9469** (XGBoost; Gini 0.8938, PR-AUC 0.9028, KS 0.7525, Brier 0.0529) on 4,862 loans never used for training or selection. NPA at the grade A-C approval rate (85.5%): 17.75% by rule, 8.12% by model, 54.3% lower (target at least 30%).	python -m src.evaluate. Tests test_auc_meets_target and test_probabilities_are_calibrated_by_decile (gap per decile at most 0.05). Limit: simulated data with a 21.8% default rate; XGBoost test AUC moves between 0.9465 and 0.9478 with the CPU thread count.
5.3 Stability	Standard deviation of the 5-fold cross-validated AUC below 0.01. **Measured: 0.0024** for XGBoost (mean 0.9497, minimum 0.9456). Logistic regression 0.0059, random forest 0.0021. Without loan_grade and loan_int_rate the test AUC is 0.9020.	python -m src.robustness (about 1 minute), reports/robustness.json. Fold seed 7, not the split seed 42.
5.4 Explainability	Every REVIEW and REJECT decision carries at least one reason (100%). **Measured: 1,201 of 1,201** REVIEW and REJECT test loans have one. The API returns up to 3. SHAP contributions plus the base value equal the model score (largest error 1.34e-05).	python -m src.explain. Test test_shap_values_add_up_to_the_score (limit 1e-3). Coverage figure from the BUILD_LOG.md summary. T2 and T3 show the reasons.
5.5 Reliability	Bad input never reaches the model: a rule failure, a missing field or a wrong type gives HTTP 422. The drift endpoint gives HTTP 409 below 100 logged rows instead of an unreliable number. The same applicant scored 0.0953 in two separate server runs. All 1,000 simulator requests returned success (the simulator stops on any error). Uptime and restart behaviour: not measured.	T4, T6, T7, T9, T11. Tests test_rejects_bad_age, test_rejects_unknown_grade, test_missing_field. SQLite writes commit as one unit or roll back and always close (src/api/store.py). Failure of the log write: not tested.
5.6 Security & Privacy	The server listens on 127.0.0.1 (the uvicorn default), so only the same laptop can reach it. The dataset has no names, addresses or phone numbers. The log keeps the submitted fields, the score, the decision, the time and a random UUID per application. Every field is type- and range-checked. Limits: no login, no TLS, no rate limit and no encryption of the SQLite file; any local process can call the API or read the log.	Review of src/api/schemas.py and src/api/store.py; T4. A dependency scan and a penetration test: not run.
5.7 Portability	Windows 10 or 11 with Python 3.11 to 3.14 and the packages pinned in requirements.txt. No Docker, no cloud, no GPU. Paths are built with pathlib; nothing is tied to a drive letter.	Built and run end to end on Windows 11 with Python 3.13.14 in a fresh virtual environment. Other Python versions, Linux and macOS: not run.

Attribute	Target Metric / Standard	Verification & Testing Method
5.8 Maintainability & Testability	13 automated tests pass (2 data, 3 model, 8 API). Each phase is one module started with python -m. Paths, feature lists, seed (42), model version and the decision cut-offs sit in src/config.py. Tests write to a temporary SQLite file, never to logs/predictions.db.	python -m pytest -q tests: 13 passed, 1 warning (T12). Code coverage: not measured.
5.9 Legal & Ethical Compliance	Public Kaggle dataset with no personal identifiers. Every decision comes with plain-language reasons, which supports the aim of the RBI Fair Practices Code that a lender can state why a loan is declined. The REVIEW band and the kill switch keep a person in the loop; the log supports audit; PSI monitoring watches the model after release. A real deployment would also fall under the DPDP Act 2023 (consent, purpose limit, retention, erasure); this prototype implements none of that. No fairness audit across age or other groups was run, and person_age is a model input.	Review of the dataset columns (notebooks/01_eda.ipynb). T2, T3 (reasons), T8, T10 (drift), T11 (kill switch). Legal review and fairness audit: not done.

Live checks referenced above (local server on 127.0.0.1:8000, 1 November 2026):

ID	Check	Result
T1	GET /health	Pass: status ok, model_version v1, kill_switch false
T2	POST /predict, good applicant (28, 42,000, RENT, grade C, 12,000 at 13.5%)	Pass: PD 0.0953, low, APPROVE; reasons RENT +0.267, income 42,000 +0.099, MEDICAL +0.051; latency 34.19 ms
T3	POST /predict, risky applicant	Pass: PD 1.0, high, REJECT; reasons loan_percent_income 0.5 (+4.2), income 18,000 (+3.686), RENT (+1.625); latency 24.01 ms
T4	POST /predict with person_age 7	Pass: HTTP 422
T5	GET /model/info	Pass: xgboost v1, 11 features, approve_below 0.2, reject_from 0.5
T6	GET /monitoring/drift with an empty log	Pass: HTTP 409
T7	500 normal applicants through /predict	Pass: p50 35.3 ms, p95 45.6 ms; APPROVE 0.718, REJECT 0.184, REVIEW 0.098
T8	Drift after normal traffic	Pass: alerts empty; window 500; all 12 rows stable (largest PSI 0.0258, model_score_PD)
T9	500 downturn applicants (incomes -20%, rates +2 points)	Pass: p50 35.1 ms, p95 44.9 ms; APPROVE 0.572, REJECT 0.274, REVIEW 0.154

ID	Check	Result
T10	Drift after downturn traffic	Pass: alerts loan_int_rate (PSI 1.1777) and decision_mix (APPROVE 0.572 against baseline 0.75); watch: person_income 0.2033, loan_percent_income 0.1234, model_score_PD 0.2371
T11	Kill switch on (SCORING_DISABLED=1), same good applicant	Pass: /health kill_switch true; PD 0.0953, band low, decision REVIEW
T12	python -m pytest -q tests	Pass: 13 passed, 1 warning (Starlette deprecation note about httpx)

6. System Design and Analysis Models

University evaluation rubrics require standard graphical analysis models illustrating the dynamic and static behavior of the application. The table lists what this SRS provides; the diagrams follow.

Diagram Type	Objective & Scope	Deliverable Format
Use Case Diagram	Maps four actors (loan applicant through a lending app, loan officer, administrator, risk analyst) to the use cases of the scoring service and the offline pipeline.	Figure 6.1 and use case specifications (Table 6.1), section 6.1.
Data Flow Diagram (DFD)	Level 0 (context): one process and four external entities. Level 1: six processes and five data stores.	Figure 2.1 (level 0) and Figure 6.2 (level 1), section 6.2.
Entity-Relationship (ERD)	The predictions table of the SQLite log and the two logical entities around it: ModelVersion (metadata and joblib) and TrainingRecord (CSV files).	Figure 6.3, Crow's Foot notation, with normalisation notes, section 6.3.
Sequence Diagram	Scoring one application: client, FastAPI, Pydantic check, pipeline, decision policy, SHAP explainer and SQLite store, with the HTTP 422 and kill-switch paths.	Figure 6.4, UML sequence, section 6.4.

6.1 Use Case Diagram

The loan applicant reaches the scoring service through a lending app. "Submit application for scoring" includes input validation, scoring and logging; "View PD, decision and reasons" includes the SHAP explanation. The loan officer handles REVIEW-band cases. The administrator checks the service health and turns on the kill switch. The risk analyst views the model info, runs the drift report, retrains and evaluates the model and runs the tests.

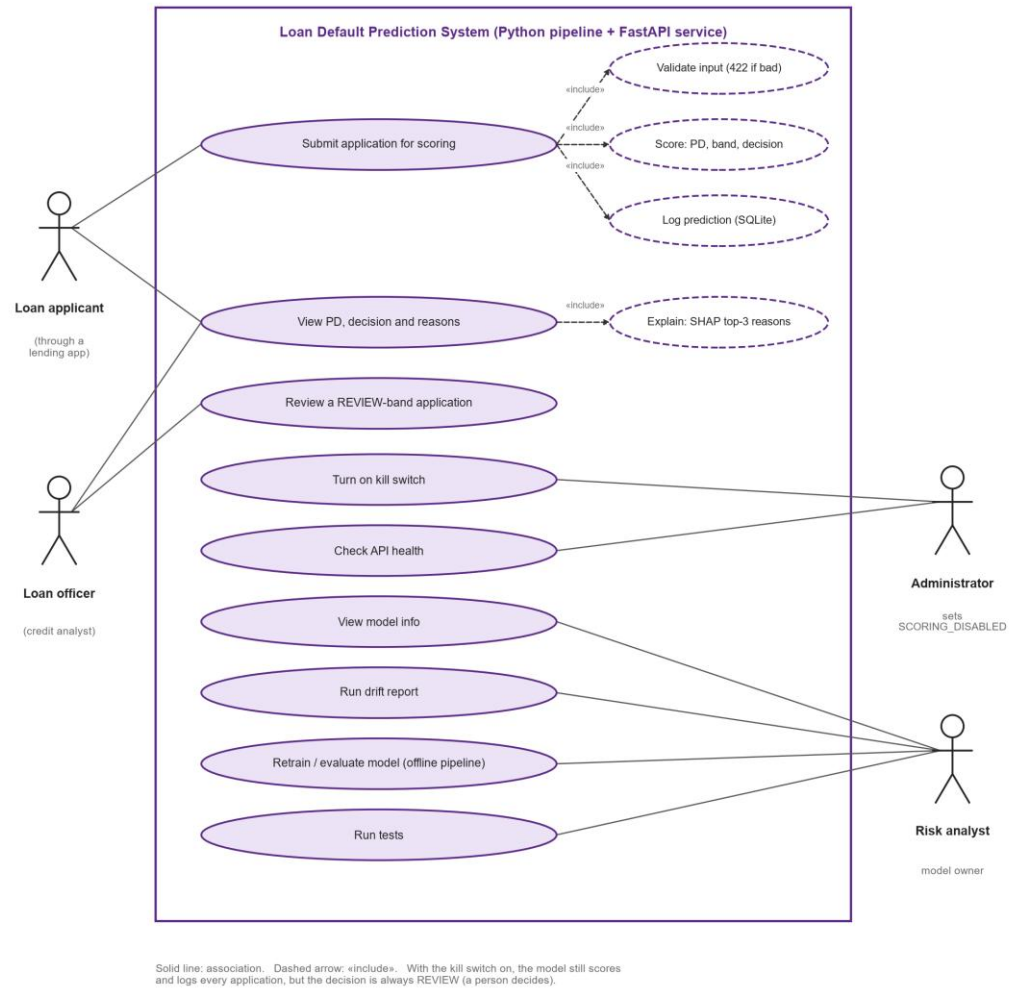


Figure 6.1: Use case diagram of the Loan Default Prediction System.

Table 6.1: Use case specifications

ID	Use Case, Actor, Requirements	Main Flow	Alternative / Error Flow
UC-1	Submit application for scoring Actor: Loan applicant, through a lending app FR-6.1 to 6.13, 8.1	1. Lending app sends POST /predict with the 10 fields. 2. Include: validate input. The API checks every field. 3. Server computes loan_percent_income. 4. Include: score. The pipeline gives PD; PD gives band and decision. 5. Include: log prediction. One row is written to SQLite. 6. API returns the JSON response.	A field is missing, out of range or of the wrong type: HTTP 422, nothing scored or logged. Kill switch on: decision is REVIEW.

ID	Use Case, Actor, Requirements	Main Flow	Alternative / Error Flow
UC-2	View PD, decision and reasons Actors: Loan applicant, Loan officer FR-5.1 to 5.5, 6.9 to 6.12	1. The app or Swagger UI shows probability_of_default, risk_band and decision. 2. Include: explain. Up to 3 SHAP reasons are listed, each with plain text. 3. The reader uses the reasons to explain the outcome to the applicant.	No factor pushes towards default: top_reasons holds fewer than 3 items, or none.
UC-3	Review a REVIEW-band application Actor: Loan officer FR-6.10, 6.11	1. The app shows decision REVIEW (PD 0.20 up to below 0.50). 2. The officer reads PD, risk band and reasons. 3. The officer decides with the applicant, outside this system.	Kill switch on: every case arrives as REVIEW. The officer's final decision is not stored (out of scope).
UC-4	Turn on kill switch Actor: Administrator FR-10.1 to 10.3	1. Stop the server. 2. Set SCORING_DISABLED=1. 3. Start the server. 4. /health shows kill_switch true; every decision is REVIEW; scoring and logging go on.	Any value other than "1": the switch stays off. Changing it needs a restart.
UC-5	Check API health Actor: Administrator FR-7.1, 7.3	1. Open /health in Swagger UI or from a script. 2. Read status, model_version and kill_switch.	Server not running: the client gets a connection error.
UC-6	View model info Actor: Risk analyst FR-7.2	1. Open /model/info. 2. Read model name, training time, features, validation metrics and the policy cut-offs.	Server not running: the client gets a connection error.
UC-7	Run drift report Actor: Risk analyst FR-9.1 to 9.7, 11.1, 11.2	1. Traffic is logged by a lending app or by python -m scripts.simulate_traffic. 2. Call /monitoring/drift?last_n=500, or run python -m src.drift. 3. The API reads the latest logged rows; PSI per feature and on the score; decision mix against the baseline. 4. The analyst reads the alerts and decides whether to retrain.	Fewer than 100 logged rows: HTTP 409 with the count.
UC-8	Retrain, evaluate and test the model Actor: Risk analyst FR-1.1 to 5.5, 12.1 to 12.3	1. Save the Kaggle CSV in data/raw. 2. Run data_prep, train, evaluate, robustness, explain and drift in order. 3. Read evaluation.json, robustness.json and the charts. 4. Run python -m pytest -q tests.	CSV missing or a column absent: an error names the file or the columns. A test fails: pytest names the failing check.

6.2 Data Flow Diagrams

The level 0 diagram is Figure 2.1. Level 1 splits the system into six processes (1.0 clean and split, 2.0 train and select model, 3.0 evaluate and back-test, 4.0 score application, 5.0 explain, 6.0 monitor

drift) and five data stores: raw CSV (D1), processed CSV (D2), model files (D3), the SQLite prediction log (D4) and the reports folder (D5). Only process 4.0 writes to D4; process 6.0 reads it.

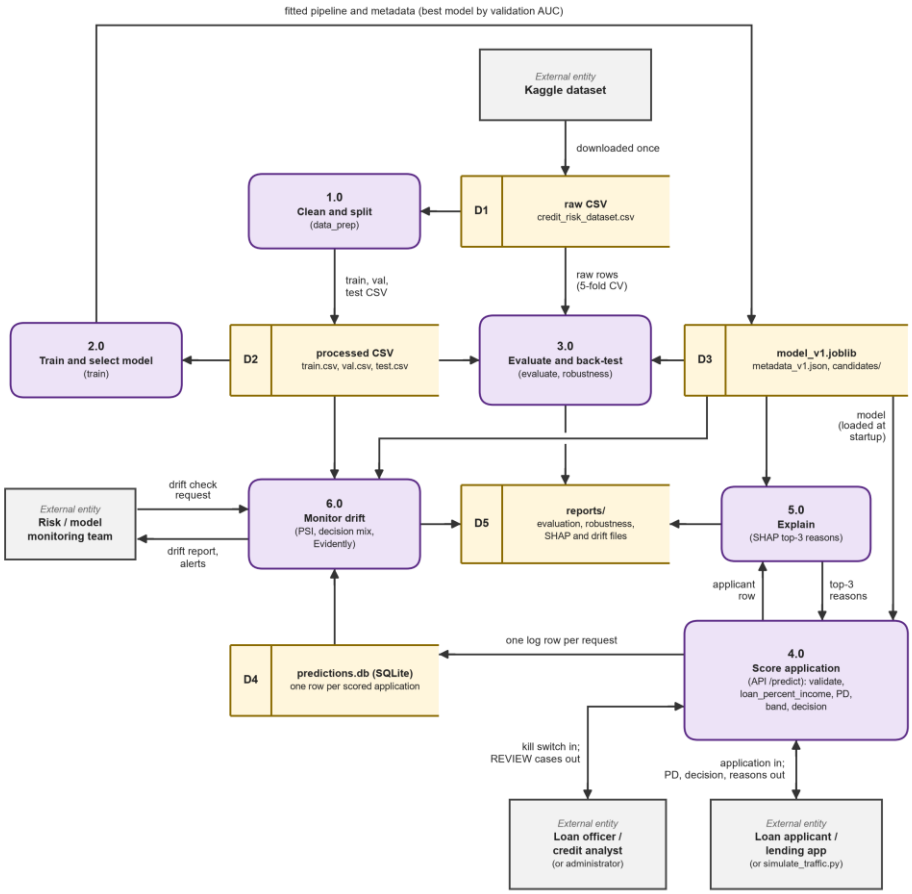


Figure 6.2: Data flow diagram, level 1. Processes 1.0 to 6.0 and data stores D1 to D5.

6.3 Entity-Relationship Diagram

Only one entity is a physical database table: predictions in the SQLite log. The other two are logical and live in files: ModelVersion (models/metadata_v1.json and model_v1.joblib) and TrainingRecord (the Kaggle CSV and the processed splits). Figure 6.3 follows the normalisation notes.

Normalisation notes.

- **Keys:** application_id (a UUID, text) is the primary key of predictions. model_version is a foreign key by value only: it names the ModelVersion that scored the row, and SQLite holds no FOREIGN KEY constraint on it. One ModelVersion scores zero or many predictions. TrainingRecord has no key column: the dataset has no id and one row is one loan. An index on created_at speeds up the "newest n rows" read.

- **Third normal form:** Every column of predictions is one atomic value and depends on application_id. A missing employment length or interest rate is stored as NULL. The risk band is not stored because it follows from pd and the two cut-offs.
- **Deliberate exceptions:** loan_percent_income is derived from loan_amnt and person_income but is stored, because it is the exact value the model saw. decision is stored although it follows from pd, because the kill switch can override it. model_version repeats in every row so that a later model v2 can be compared with v1.
- **Logical model:** The log denormalises on purpose: inputs, score and decision sit in one row so that drift monitoring and an audit can replay a decision without joins. The applicant is not identified; the row holds no name or contact detail.

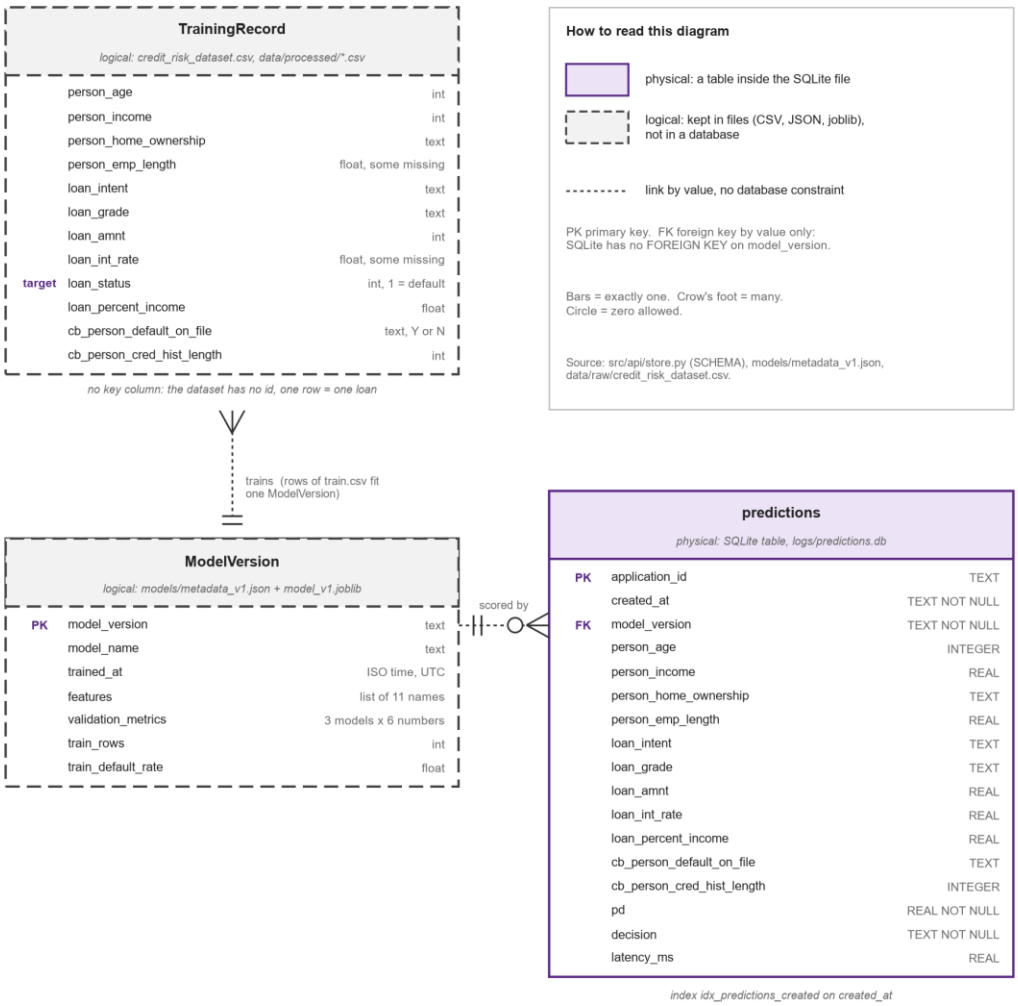


Figure 6.3: Entity-relationship diagram (Crow's Foot notation).

6.4 Sequence Diagram

The sequence shows the main feature of the system, scoring one application with POST /predict. The break frame is the error path when a field fails the Pydantic check: the API answers HTTP 422 and the pipeline is never called. The opt frame is the kill switch: with SCORING_DISABLED=1 the decision policy sets the decision to REVIEW. The timer behind latency_ms starts after validation and stops before the log write.

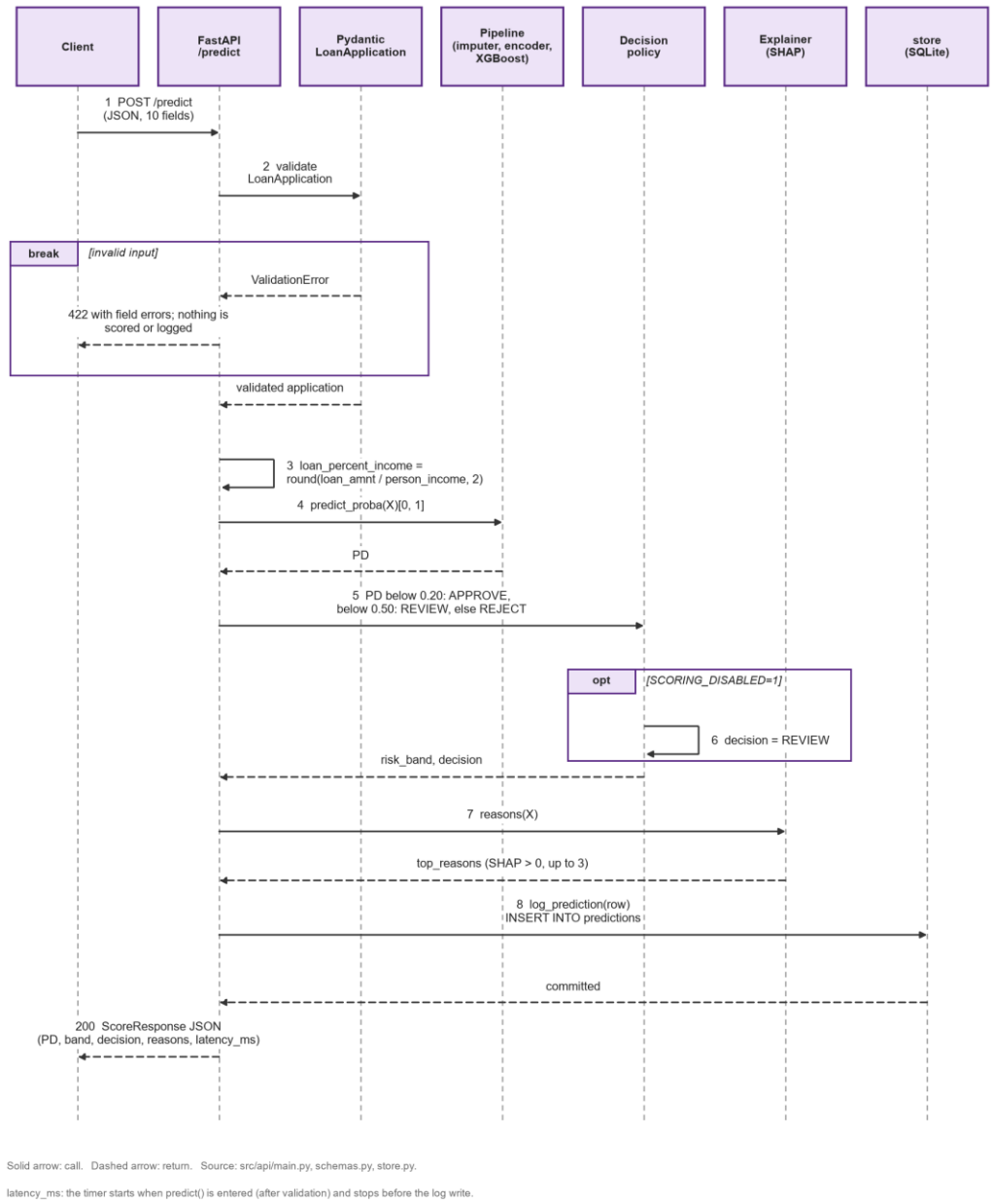


Figure 6.4: Sequence diagram, scoring one loan application.

7. Academic Review & Sign-Off

This Software Requirements Specification document for Loan Default Prediction System: Explainable Credit Risk Scoring with Drift Monitoring has been submitted and reviewed by the academic supervisory panel:

Project Coordinator / Guide	Internal Examiner	Head of Department (HOD)
Signature: _____ Date: _____	Signature: _____ Date: _____	Signature: _____ Date: _____